Defining Relations
DDL statements

Creating Relations in SQL

- Creates the Students relation. Observe that the type (domain) of each field is specified, and enforced by the DBMS whenever tuples are added or modified (domain constraints)
- As another example, the Enrolled table holds information about courses that students take.

```
CREATE TABLE Students
        (sid: CHAR(20),
         name: CHAR(20),
         login: CHAR(10),
         age: INTEGER,
         gpa: REAL)
```

```
CREATE TABLE Enrolled
        (sid: CHAR(20),
         cid: CHAR(20),
         grade: CHAR(2))
```

Destroying and Altering Relations

DROP TABLE Students

- Destroys the relation Students. The schema information *and* the tuples are deleted.

ALTER TABLE Students
    ADD COLUMN firstYear: integer

❖ The schema of Students is altered by adding a new field; every tuple in the current instance is extended with a *null* value in the new field.

Adding and Deleting Tuples

Can insert a single tuple using:

```
INSERT INTO  Students (sid, name, login, age, gpa)
VALUES  (53688, 'Smith', 'smith@ee', 18, 3.2)
```

Can delete all tuples satisfying some condition (e.g., name = Smith):

```
DELETE
FROM Students S
WHERE S.name = 'Smith'
```

Integrity Constraints (ICs)

- IC: condition that must be true for any instance of the database; e.g., <u>domain constraints</u>
  - ICs are specified when schema is defined
  - ICs are checked when relations are modified
- A legal instance of a relation is one that satisfies all specified ICs
  - DBMS should not allow illegal instances
- If the DBMS checks ICs, stored data is more faithful to real-world meaning
  - Avoids data entry errors, too!

Primary Key Constraints

- A set of fields is a *key* for a relation if :
  1. No two distinct tuples can have same values in all key fields, and
  2. This is not true for any subset of the key.
  - Part 2 false? A *superkey*.
  - If there's >1 key for a relation, one of the keys is chosen (by DBA) to be the *primary key*.
- E.g., *sid* is a key for Students. (What about *name*?) The set {*sid, gpa*} is a superkey.

Primary and Candidate Keys in SQL

- Possibly many *candidate keys* (specified using UNIQUE), one of which is chosen as the *primary key*.

❖ "For a given student and course, there is a single grade." vs. "Students can take only one course, and receive a single grade for that course; further, no two students in a course receive the same grade."

❖ Used carelessly, an IC can prevent the storage of database instances that arise in practice!

```
CREATE TABLE Enrolled
  (sid CHAR(20)
   cid  CHAR(20),
   grade CHAR(2),
   PRIMARY KEY  (sid,cid) )
```

```
CREATE TABLE Enrolled
  (sid CHAR(20)
   cid  CHAR(20),
   grade CHAR(2),
   PRIMARY KEY  (sid),
   UNIQUE (cid, grade) )
```

Foreign Keys, Referential Integrity

- *Foreign key* : Set of fields in one relation that is used to `refer' to a tuple in another relation. (Must correspond to primary key of the second relation.) Like a `logical pointer'.
- E.g. *sid* is a foreign key referring to Students:
  - Enrolled(*sid*: string, *cid*: string, *grade*: string)
  - If all foreign key constraints are enforced, *referential integrity* is achieved

Foreign Keys in SQL

- Only students listed in the Students relation should be allowed to enroll for courses

```
CREATE TABLE Enrolled
  (sid CHAR(20),  cid CHAR(20),  grade CHAR(2),
   PRIMARY KEY  (sid,cid),
   FOREIGN KEY (sid) REFERENCES Students )
```

Enrolled

sid	cid	grade
53666	Carnatic101	C
53666	Reggae203	B
53650	Topology112	A
53666	History105	B

Students

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

Enforcing Referential Integrity

- Consider Students and Enrolled; *sid* in Enrolled is a foreign key that references Students
- What should be done if an Enrolled tuple with a non-existent student id is inserted? (*Reject it!*)
- What should be done if a Students tuple is deleted?
  - Also delete all Enrolled tuples that refer to it
  - Disallow deletion of a Students tuple that is referred to
  - Set sid in Enrolled tuples that refer to it to a *default sid*
  - (In SQL, also: Set sid in Enrolled tuples that refer to it to a special value *null,* denoting *`unknown'* or *`inapplicable'*) but sid is part of a primary key.
  - Default is NO ACTION ie DELETE or UPDATE is rejected
- **Similar if primary key value of Students tuple is updated**

Referential Integrity in SQL

- SQL/92 and SQL:1999 support all 4 options on deletes and updates
  - Default is NO ACTION (*delete/update is rejected*)
  - CASCADE (also delete all tuples that refer to deleted tuple)
  - SET NULL / SET DEFAULT (sets foreign key value of referencing tuple)
  - sid (CHAR (20) DEFAULT ‘53995’

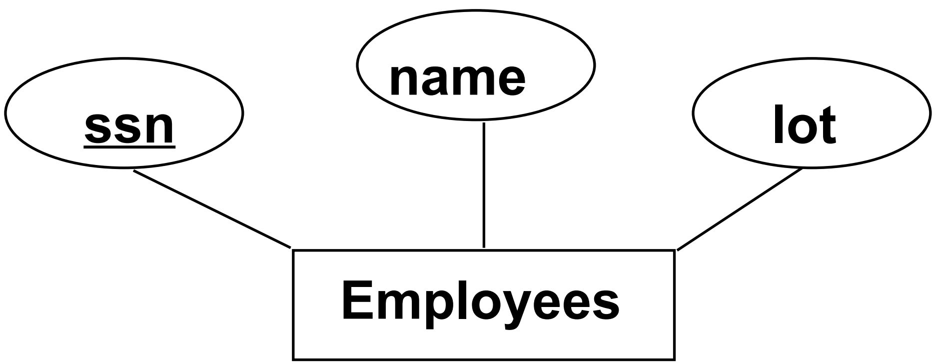
```
CREATE TABLE Enrolled
  (sid CHAR(20),
   cid CHAR(20),
   grade CHAR(2),
   PRIMARY KEY  (sid,cid),
   FOREIGN KEY (sid)
     REFERENCES Students
        ON DELETE CASCADE
        ON UPDATE SET DEFAULT )
```

Where do ICs Come From?

- ICs are based upon the semantics of the real-world enterprise that is being described in the database relations
- We can check a database instance to see if an IC is violated, but we can NEVER infer that an IC is true by looking at an instance
  - An IC is a statement about *all possible* instances!
- Key and foreign key ICs are the most common; more general ICs supported too

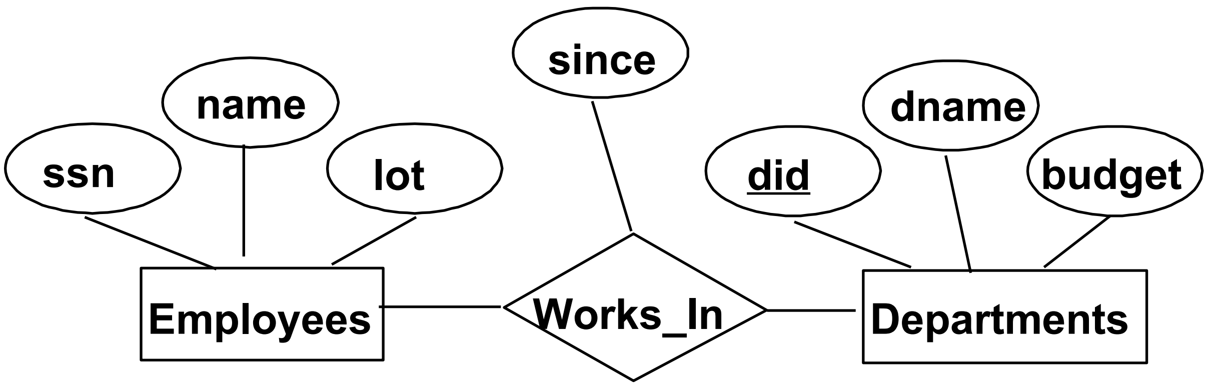
Logical DB Design: ER to Relational

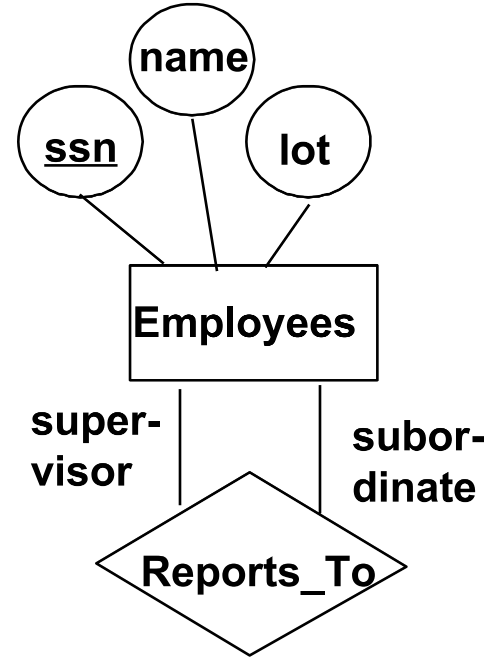
- Entity sets to tables:




```
CREATE TABLE Employees
      (ssn CHAR(11),
      name CHAR(20),
      lot  INTEGER,
      PRIMARY KEY  (ssn))
```

ER Model Basics (2)





- *Relationship*: Association among two or more entities. E.g., Rajendra works in research & development department.
- *Relationship Set*: Collection of similar relationships.
  - An n-ary relationship set R relates n entity sets E1 ... En; each relationship in R involves entities e1 € E1, ..., en € En
    - Same entity set could participate in different relationship sets, or in different “roles” in same set.

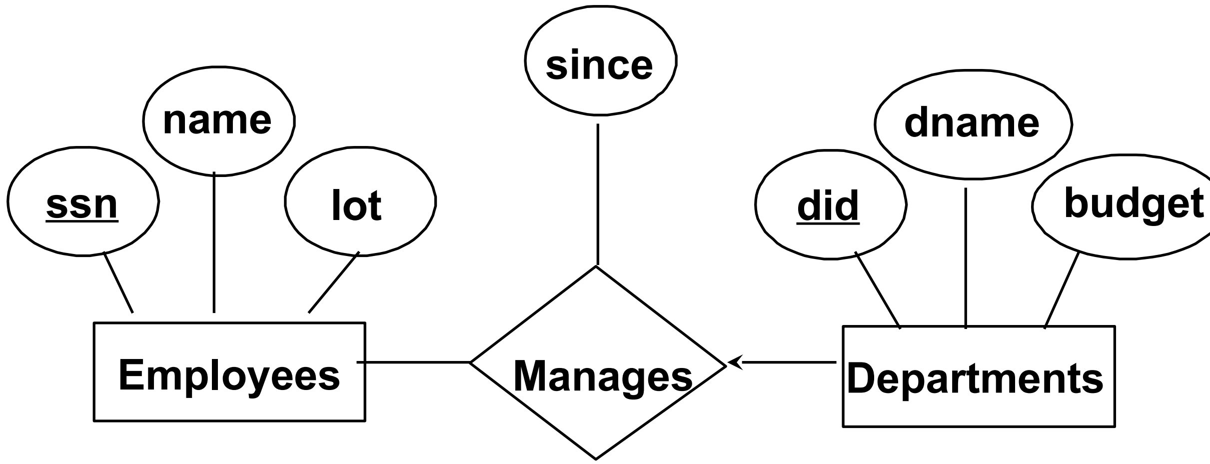
Relationship Sets to Tables

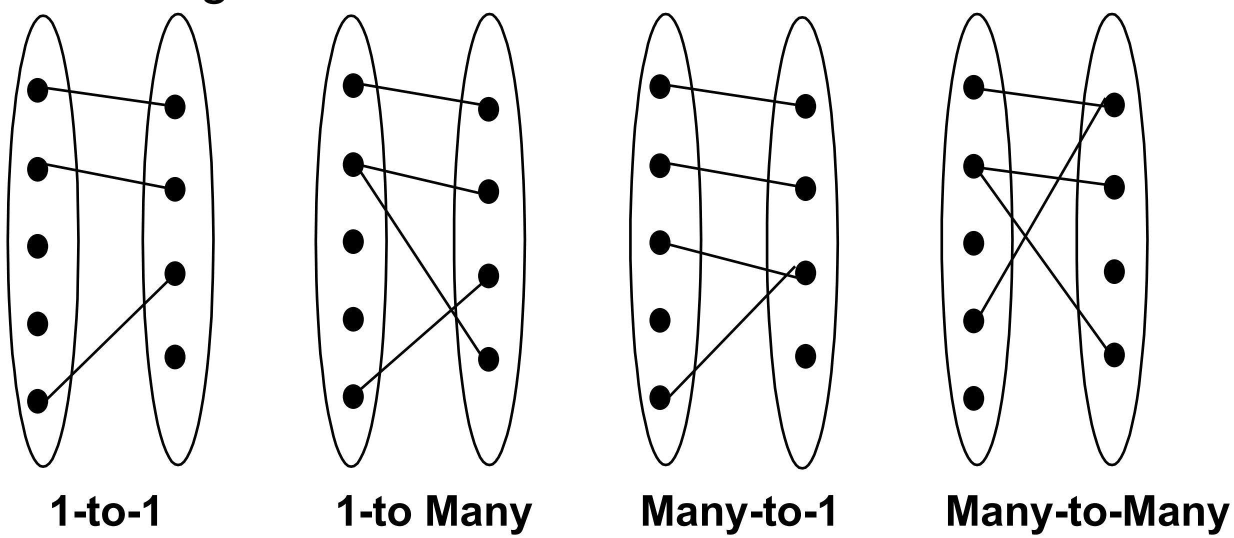
- In translating a relationship set to a relation/table, attributes of the relation must include:
  - Keys for each participating entity set (as foreign keys)
    - This set of attributes forms a superkey for the relation
  - All descriptive attributes

```
CREATE TABLE Works_In(
  ssn  CHAR(11),
  did  INTEGER,
  since  DATE,
  PRIMARY KEY (ssn, did),
  FOREIGN KEY (ssn)
       REFERENCES Employees,
  FOREIGN KEY (did)
       REFERENCES Departments)
```

Review: Key Constraints

- Each dept has at most one manager, according to the *key constraint* on Manages.





Translation to relational model?

Department-Manages-Employee

```
CREATE TABLE  Departments
            (dname  CHAR(20),
            did  INTEGER,
          budget INTEGER,
            PRIMARY KEY  (did))


 CREATE TABLE Employees
          (ssn CHAR(11),
          name CHAR(20),
          lot  INTEGER,
          PRIMARY KEY  (ssn))
```

Translating ER Diagrams with Key Constraints

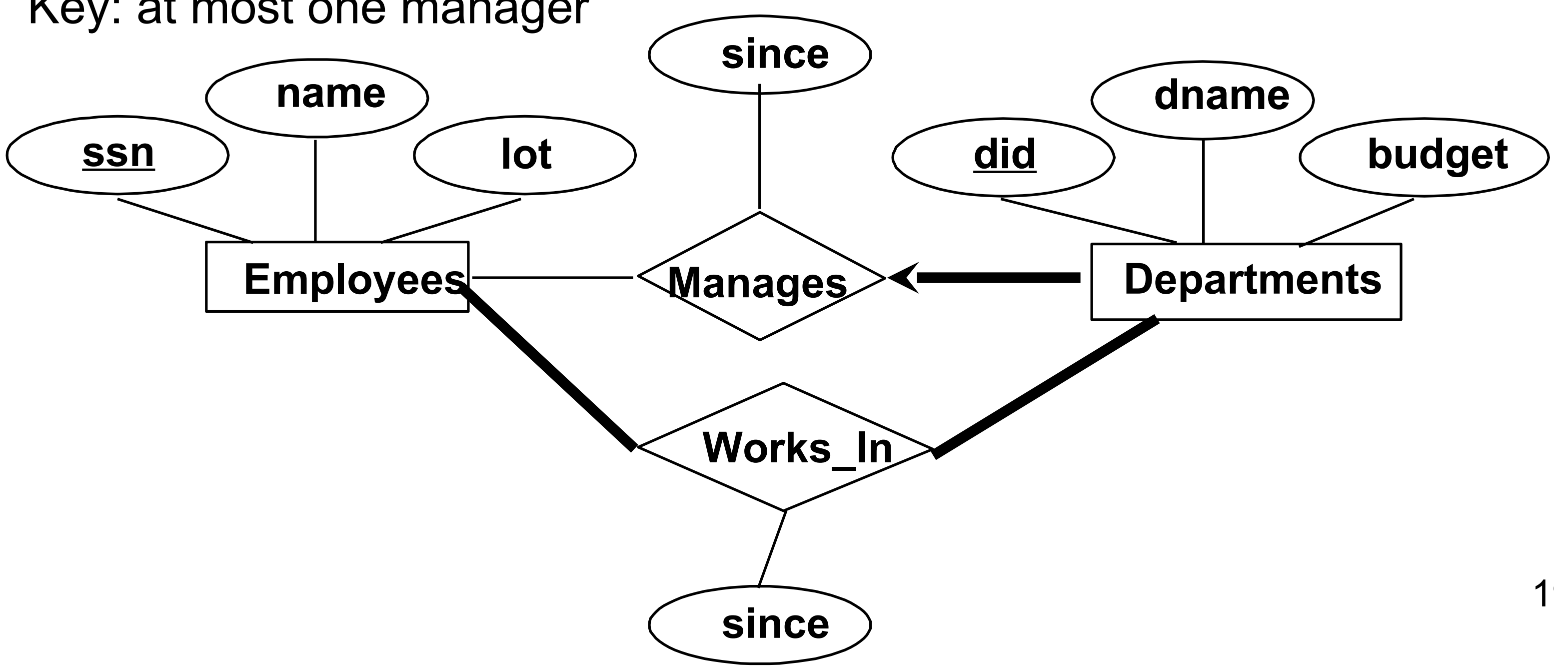
- Map relationship to a table:
  - Note that did is the key now!
  - Separate tables for Employees and Departments
- Since each department has a unique manager, we could instead combine Manages and Departments

```
CREATE TABLE Manages(
  ssn CHAR(11),
  did INTEGER,
  since DATE,
  PRIMARY KEY (did),
  FOREIGN KEY (ssn) REFERENCES Employees,
  FOREIGN KEY (did) REFERENCES Departments)
```

```
CREATE TABLE Dept_Mgr(
  did INTEGER,
  name CHAR(20),
  budget REAL,
  ssn CHAR(11),
  since DATE,
  PRIMARY KEY (did),
  FOREIGN KEY (ssn) REFERENCES Employees)
```

Review: Participation Constraints

- Does every department have a manager?
  - If so, this is a *participation constraint*: the participation of Departments in Manages is said to be *total* (vs. *partial*).
    - Every *did* value in Departments table must appear in a row of the Manages table (with a non-null *ssn* value!)
    - Participation: every dept has manager
    - Key: at most one manager



Participation Constraints in SQL

- We can capture participation constraints involving one entity set in a binary relationship

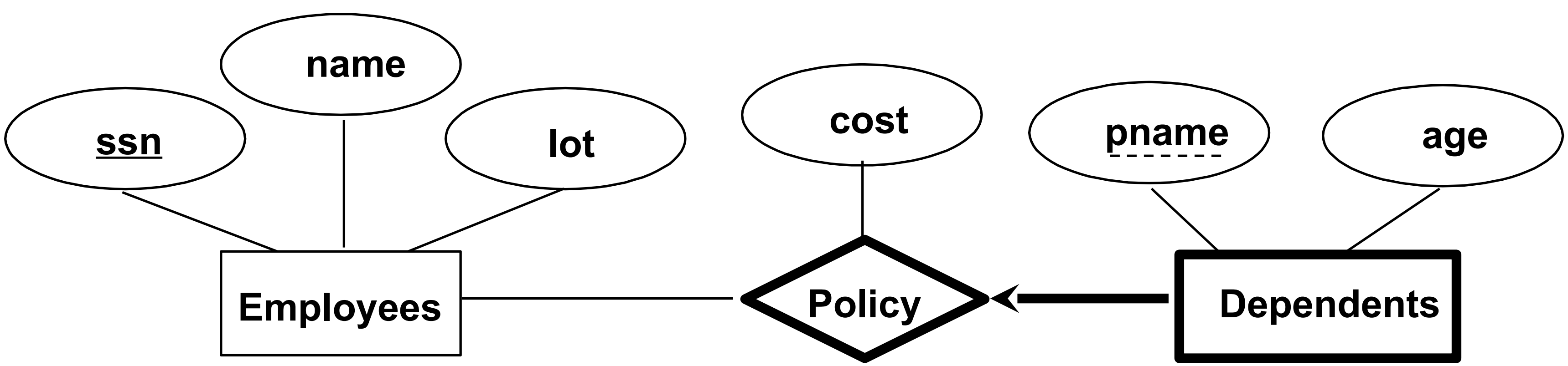
```
CREATE TABLE  Dept_Mgr(
  did  INTEGER,
  dname  CHAR(20),
  budget  REAL,
  ssn  CHAR(11) NOT NULL,
  since  DATE,
  PRIMARY KEY  (did),
  FOREIGN KEY  (ssn) REFERENCES Employees,
  ON DELETE NO ACTION)
```

Contd…

- On delete no action ….is default and need not be explicitly specified
- Ensures that an Employee tuple can not be deleted while it is pointed to by a Dept-Mgr tuple
- If we wish to delete that Employee tuple we first need to have a new employee as manager in Dept-Mgr
- If we specify CASCADE instead of NO ACTION
  - Deleting information about the department itself because the manager has been fired !!!!
- The constraint that every dept has a manager cant be captured using first option (separate manages relation option)
- If NOTNULL is added to ssn and did
  - It would prevent firing a manager

Review: Weak Entities

- A *weak entity* can be identified uniquely only by considering the primary key of another (*owner*) entity.
  - Owner entity set and weak entity set must participate in a one-to-many relationship set (1 owner, many weak entities).
  - Weak entity set must have total participation in this *identifying* relationship set.



Translating Weak Entity Sets

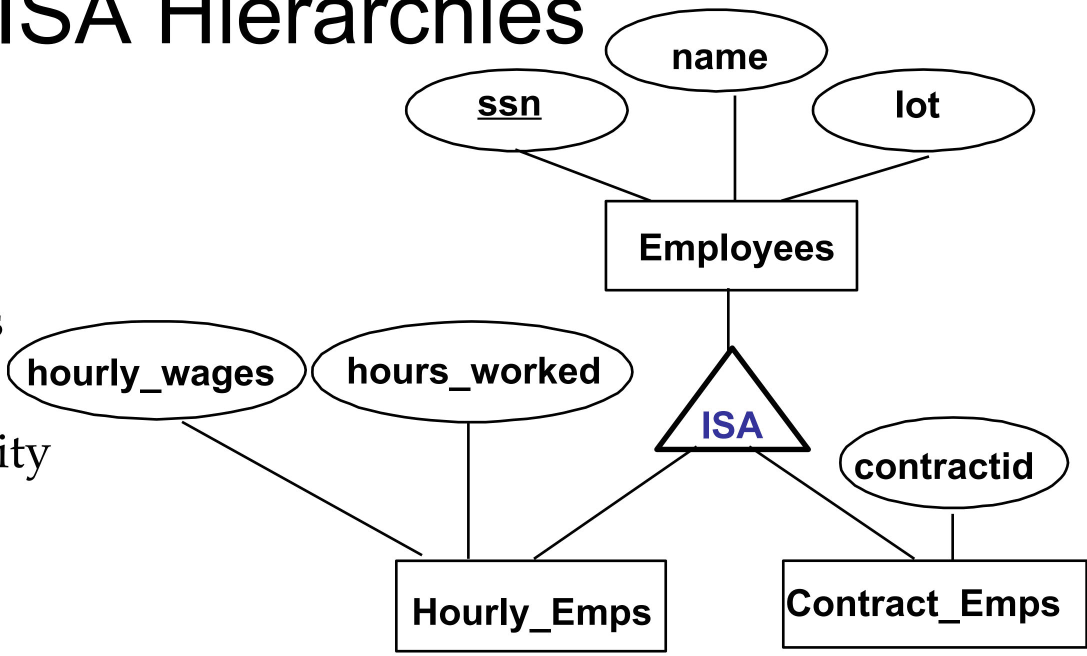
- Weak entity set and identifying relationship set (course) are translated into a single table.
  - When the owner entity is deleted, all owned weak entities must also be deleted.

```
CREATE TABLE  Dep_Policy (
  pname  CHAR(20),
  age  INTEGER,
  cost  REAL,
  ssn  CHAR(11) NOT NULL,
  PRIMARY KEY  (pname, ssn),
  FOREIGN KEY  (ssn) REFERENCES Employees,
    ON DELETE CASCADE)
```

Reports-To Relationship Set

```
CREAT TABLE Reports-To(
supervisor_ssn CHAR(11),
subordinate_ssn CHAR(11),
PRIMARY KEY (supervisor_ssn,subordinate_ssn),
FOREIGN KEY (supervisor_ssn) REFERENCES Employees (ssn),
FOREIGN KEY (subordinate_ssn) REFERENCES Employees (ssn))
```

Review: ISA Hierarchies



❖ As in C++, or other PLs, attributes are inherited.

❖ If we declare A **ISA** B, every A entity is also considered to be a B entity.

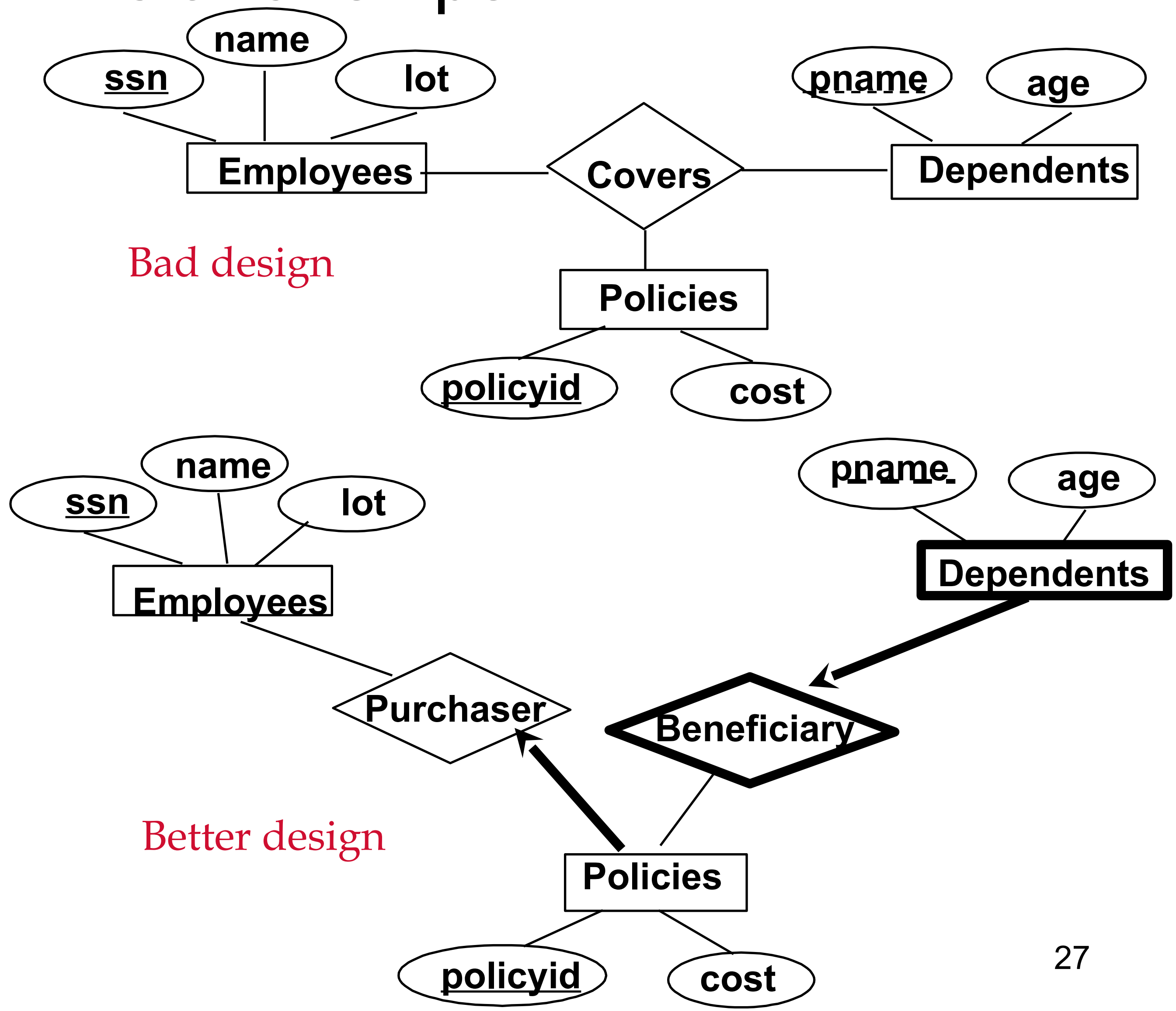
- *Overlap constraints*: Can Joe be an Hourly_Emps as well as a Contract_Emps entity? (*Allowed/disallowed*)
- *Covering constraints*: Does every Employees entity also have to be an Hourly_Emps or a Contract_Emps entity? *(Yes/no)*

Translating ISA Hierarchies to Relations

- ***General approach:***
  - 3 relations: Employees, Hourly_Emps and Contract_Emps.
    - *Hourly_Emps*:
      - Every employee is recorded in Employees For hourly emps, extra info recorded in Hourly_Emps (*hourly_wages*, *hours_worked*, *ssn*)
      - must delete Hourly_Emps tuple if referenced Employees tuple is deleted)
    - Queries involving all employees easy, those involving just Hourly_Emps require a join to get some attributes.
- Alternative: Just Hourly_Emps and Contract_Emps.
  - *Hourly_Emps*: *ssn*, *name, lot, hourly_wages, hours_worked.*
  - Each employee must be in one of these two subclasses.

Review: Binary vs. Ternary Relationships

- What are the additional constraints in the 2nd diagram?



Translating to Relational Model

- Deletion of Employee leads to deletion of all policies owned by employee and all dependents who are beneficiaries of those policies
- Each dependent is required to have a covering policy (implicit NOT NULL)

Binary vs. Ternary Relationships (Contd.)

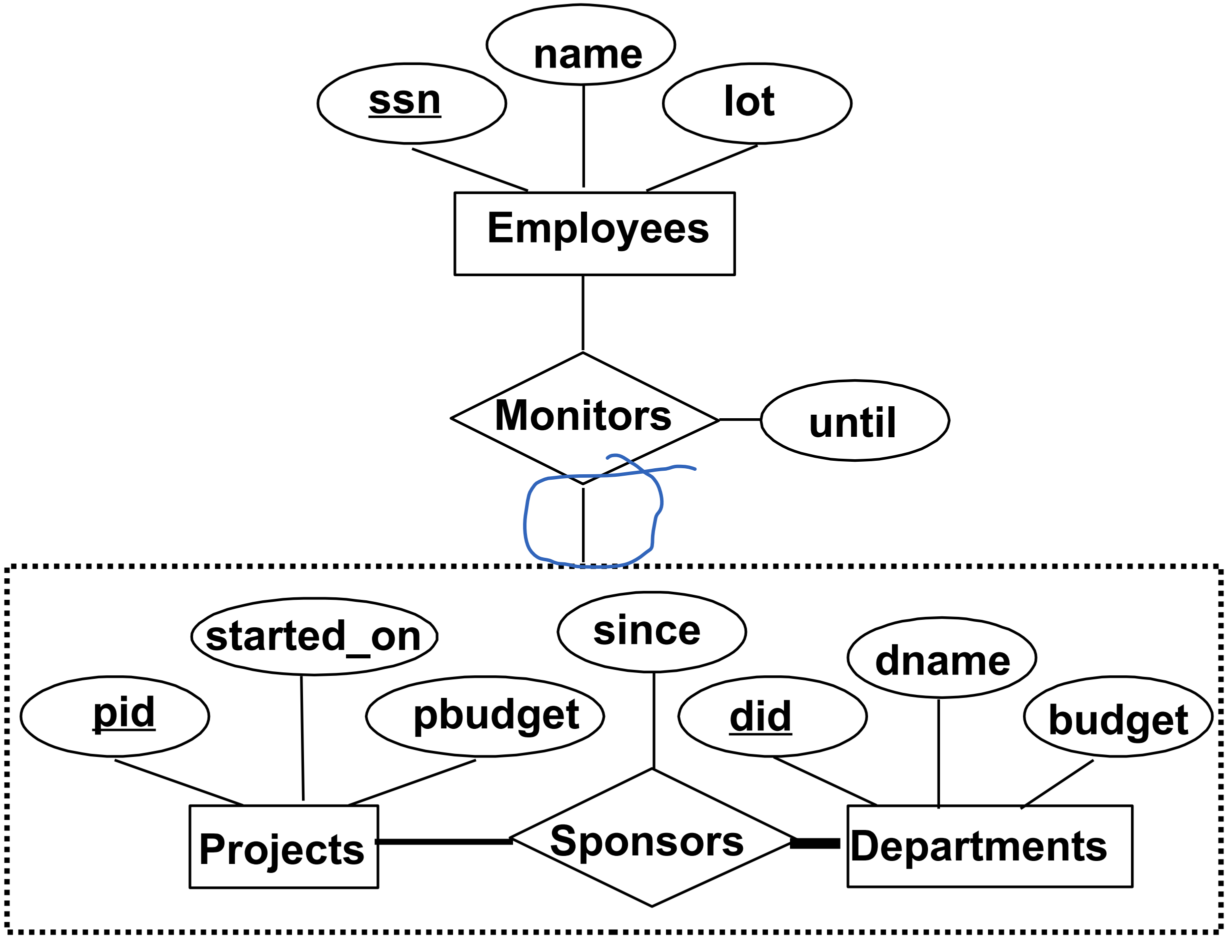
- The key constraints allow us to combine Purchaser with Policies and Beneficiary with Dependents.
- Participation constraints lead to NOT NULL constraints.
- What if Policies is a weak entity set?

```
CREATE TABLE  Policies (
  policyid  INTEGER,
  cost  REAL,
  ssn  CHAR(11)  NOT NULL,
  PRIMARY KEY (policyid).
  FOREIGN KEY (ssn) REFERENCES Employees,
    ON DELETE CASCADE)

CREATE TABLE Dependents (
  pname  CHAR(20),
  age  INTEGER,
  policyid  INTEGER,
  PRIMARY KEY (pname, policyid).
  FOREIGN KEY (policyid) REFERENCES Policies,
    ON DELETE CASCADE)
```

Aggregation

- Used when we have to model a relationship involving (entitity sets and) a relationship set
  - Aggregation allows us to treat a relationship set as an entity set for purposes of participation in (other) relationships



☛ *Aggregation vs. ternary relationship*:

❖ Monitors is a distinct relationship, with a descriptive attribute

❖ Also, can say that each sponsorship is monitored by at most one employee

Aggregation

- Projects, Departments, Sponsors translated as discussed
  - Projects(pid, started_on, pbudget)
  - Departments (did, dname, budget)
  - Sponsors (pid,did,since)
- Monitors (until, pid, did, ssn)
- Employee (ssn, name, lot)

- Why Sponsors is required?
  - To record descriptive attributes of Sponsor (since)
  - Not every sponsorship has a monitor and thus some (pid,did) pairs in sponsors may not appear in Monitors
- If Sponsors
  - has no descriptive attributes
  - Has total participation in Monitors
- Then every possible instance of Sponsors can be obtained from (pid,did) columns of Monitors (Sponsors can be dropped)

Views

- A *view* is just a relation, but we store a *definition*, rather than a set of tuples]
- From students and Enrolled relations

```
CREATE  VIEW  YoungActiveStudents (name, grade)
        AS  SELECT   S.name, E.grade
        FROM  Students S, Enrolled E
        WHERE  S.sid = E.sid and S.age<21
```

- Views can be dropped using the DROP VIEW command
  - How to handle DROP TABLE if there's a view on the table?
    - DROP TABLE command has options to let the user specify this

Updating Views

- Updates to be specified only on views that are defined on a single base table (SQL 92) using just projection and selection, with no use of aggregate operations (updatable views)

  CREATE TABLE Good Students (sid,gpa)

  AS SELECT S.sid, S.gpa

  FROM Students S

  WHERE S.gpa > 9.0

  If view does not contain key of the base table, several rows in table would correspond to a single row in view

- SQL 99 allows updates on views using more than one table where the PK of that table is included in the fields of view
- Views defined using UNION, INTERSECT, EXCEPT cant be inserted into even if they are updatable

Insertion

- If we insert a GoodStudents row by inserting a row into Students, using null values in columns of students that do not appear in GoodStudents(eg sams,login)
- Note that PK columns are not allowed to contain null values
- Therefore if we attempt to insert rows through a view that does not contain the PK of the underlying table, the insertions will be rejected

Views and Security

- Views can be used to present necessary information (or a summary), while hiding details in underlying relation(s)
- Given YoungStudents, but not Students or Enrolled, we can find students s who are enrolled, but not the *cid's* of the courses they are enrolled in

Relational Model: Summary

- A tabular representation of data
- Simple and intuitive, currently the most widely used
- Integrity constraints can be specified by the DBA, based on application semantics. DBMS checks for violations
  - Two important ICs: primary and foreign keys
  - In addition, we *always* have domain constraints
- Powerful and natural query languages exist
- Rules to translate ER to relational model